

Microservice, front-end and back-end

Source document

It contains the how to develop the microservice architecture and the back-end.

The controller in the back-end

```
/**
 * (Picture)
 **/
@RestController
@RequestMapping("picture")
@CrossOrigin
public class PictureController {

    @Autowired
    private PictureDao pictureDao;

    @Autowired
    private UserDao userDao;

    @Autowired
    private WebSocket websocket;
    /**
     *
     */
    @Resource
    private PictureService pictureService;

    @PostMapping("search")
    public R search(String searchText) {
        System.out.println(searchText);
        websocket.sendMessage("Search successfully");
        List<Picture> pictures = pictureDao.selectList(new QueryWrapper<Picture>()
            .eq("type",0)
            .like(searchText != null && !searchText.equals(""), "name", searchText));
        return R.success(pictures);
    }
}
```

```

    }

    @GetMapping("startDetection")
    public R startDetection() {

        //todo call the analysis algorithm
        VideoUtil.fun();

        return R.success(null);
    }

    /**
     * Search the data by key
     * Key
     * @return data
     */
    @GetMapping("getAllVideo")
    public R getAllVideo(){
        List<Picture> pictureList = this.pictureService.list(new
        QueryWrapper<Picture>().eq("type", 1));
        return R.success(pictureList);
    }

    @PostMapping("register")
    public R register(User user){

        user.setName(user.getUsername());
        userDao.insert(user);
        return R.success(user);
    }

    @PostMapping("login")
    public R login(User user){

        User user1 = userDao.selectOne(new QueryWrapper<User>().eq("username",
        user.getUsername())
        .eq("password", user.getPassword()));
        if(user1!=null)
            return R.success(user);
        else
            return R.failed();
    }

```

```

@PostMapping("send")
public R send(){
    websocket.sendMessage("Call ");
    return R.success(null);
}

/**
 * Delete
 *
 * @param idList
 * @return Delete
 */
@DeleteMapping
public R delete(@RequestParam("idList") List<Long> idList) {
    return R.success(this.pictureService.removeByIds(idList));
}

// @PostMapping("login")
// public R login(String username,String password){
//     System.out.println(username+"/"+password);
//     return R.success(null);
// }

@PostMapping("uploadImg")
public R img(@RequestParam("file") MultipartFile file) throws IOException {

    if (file.isEmpty()) {
        return R.failed();
    }

    //File name
    String fileName = file.getOriginalFilename();
    //Allow the file name contain the suffix
    List<String> FILE_WHILE_EXT_LIST = Arrays.asList("JPG","PNG","JPEG","GIF");
    Assert.notNull(fileName,"File name can not be empty");
    String fileExtName = fileName.substring(fileName.lastIndexOf(".") + 1);
    if (FILE_WHILE_EXT_LIST.contains(fileExtName.toUpperCase())){
        //location of file
        //SAVE_PATH  E:/liangd/Java/stmg-front/src/main/webapp
        //IMG_PATH  /upload/
        fileName = UUID.randomUUID()+ "." +fileExtName;

        URL url = ResourceUtils.getURL("classpath:");
    }
}

```

```

        String r = url.getPath();
        OutputStream outputStream = new FileOutputStream(new File(r+ "static/"
+fileName).getPath());
        outputStream.write(file.getBytes());

        Picture picture = new Picture();
        picture.setName(fileName);
        picture.setPath( fileName);
        picture.setType(0);
        pictureDao.insert(picture);

        return R.success(picture);
    }

    @PostMapping("uploadVideo")
    public R video(@RequestParam("file") MultipartFile file) throws IOException {
        if (file.isEmpty()) {
            return R.failed();
        }
        //
        String fileName = file.getOriginalFilename();
        Assert.notNull(fileName,"File name can not be empty");
        URL url = ResourceUtils.getURL("classpath:");
        String r = url.getPath();
        OutputStream outputStream = new FileOutputStream(new File(r+ "static/"
+fileName).getPath());
        outputStream.write(file.getBytes());

        Picture picture = new Picture();
        picture.setName(fileName);
        picture.setPath( ""+fileName);
        picture.setType(1);
        pictureDao.insert(picture);

        return R.success(picture);
    }

}

```

The component of achieving the message communication

This component is used to send and response the message.

```
package com.video.controller;

public class R<T> {
    private int code;
    private String msg;
    private T body;

    public R() {
    }

    public R(int code, String msg, T body) {
        this.code = code;
        this.msg = msg;
        this.body = body;
    }

    public static R failed() {
        return new R(-1,"fail",null);
    }

    public static <T> R success(T t) {
        return new R(0,"success",t);
    }

    public int getCode() {
        return code;
    }

    public void setCode(int code) {
        this.code = code;
    }

    public String getMsg() {
        return msg;
    }

    public void setMsg(String msg) {
        this.msg = msg;
    }
}
```

```

    public T getBody() {
        return body;
    }

    public void setBody(T body) {
        this.body = body;
    }
}

```

The WebSocket define

It contains the foundation of microservice

```

package com.video.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import
org.springframework.web.socket.config.annotation.AbstractWebSocketMessageBrokerConfigurer;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.server.standard.ServerEndpointExporter;
import javax.websocket.server.ServerEndpoint;

```

@Configuration

```

public class WebSocketConfig extends AbstractWebSocketMessageBrokerConfigurer {

    @Bean
    public ServerEndpointExporter serverEndpointExporter(){
        return new ServerEndpointExporter();
    }
}

```

<!DOCTYPE HTML>

<html>

<head>

<title>Websocket</title>

<meta charset="UTF-8">

<link

rel="stylesheet"

href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"

```

integrity="sha384-
ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
crossorigin="anonymous">
    <script      src="https://code.jquery.com/jquery-3.3.1.slim.min.js"      integrity="sha384-
q8i/X+965DzO0rT7abK41JStQIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo"
crossorigin="anonymous"></script>
    <script      src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.7/umd/popper.min.js"
integrity="sha384-
UO2eT0CpHqdSJQ6hJty5KVphtPhzWj9WO1clHTMGa3JDZwrnQq4sF86dIHNDz0W1"
crossorigin="anonymous"></script>
    <script      src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js"
integrity="sha384-
JjSmVgyd0p3pXB1rRibZUAYoIIy6OrQ6VrjIEaFf/nJGzIxFDsf4x0xIM+B07jRM"
crossorigin="anonymous"></script>
    <script src="https://code.jquery.com/jquery-3.1.1.min.js"></script>
</head>

```

```

<body>
<div class="progress" style="width: 60%;height: 60px;margin: 0 auto;margin-top: 150px;">
    <div class="progress-bar" role="progressbar" style="width: 25%" aria-valuenow="25" aria-
valuemin="0" aria-valuemax="100"></div>
</div>
<div id="message"></div>
<input type="button" name="" id="btn" value="start" />
</body>

```

```

<script type="text/javascript">
    var websocket = null;

    //judge the web whether could use the WebSocket
    if('WebSocket' in window){
        websocket = new WebSocket("ws://localhost:8080/websocket");
    }else{
        alert('Not support websocket')
    }

    //return fail when fail to connect
    websocket.onerror = function(){
        setMessageInnerHTML("fail to connect");
    };

    //Connect successfully
    websocket.onopen = function(event){
        setMessageInnerHTML("connected");
    };

```

```

    }

    //receive the message
    websocket.onmessage = function(event){
        console.log(event.data)
        setMessageInnerHTML(event.data)
        // setMessageInnerHTML(event.data);
        $(".progress-bar").attr("style","width:"+event.data+"%")
    }

    //close connection
    websocket.onclose = function(){
        setMessageInnerHTML("close");
    }

    // Listen for window closure events. When the window is closed, actively close the websocket
    connection to prevent closing the window before the connection is disconnected, and the server will
    throw an exception.
    window.onbeforeunload = function(){
        alert("shut down");
        websocket.close();
    }

    // Display messages on web pages
    function setMessageInnerHTML(innerHTML){
        document.getElementById('message').innerHTML = innerHTML ;
    }

    //Close the connection
    function closeWebSocket(){
        alert("Close the connection ");
        websocket.close();
    }

    //Start
    $("#btn").click(function(){
        $.ajax({
            url: "http://localhost:8080/picture/send",
            type:'post',
            success: function(HTML) {//return the content
                console.log(HTML);
            }
        });
    })
</script>
</html>

```

Database used in the system

server:

port: 8080

spring:

servlet:

multipart:

max-file-size: 1000MB

max-request-size: 1000MB

session:

timeout: 30m

datasource:

driver-class-name: com.mysql.cj.jdbc.Driver

url: jdbc:mysql://localhost:3306/video_detection

username: root

password: 123456

The front-end Ui design

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>video analysis system</title>
```

```
  <link rel="stylesheet" href="https://unpkg.com/element-ui/lib/theme-chalk/index.css">
```

```
  <style>
```

```
    html,body{
```

```
      height: 100%;
```

```
      padding: 0px;
```

```
      margin: 0px;
```

```
    }
```

```
    #app{
```

```
      height: 100%;
```

```
      padding: 50px;
```

```
      margin: 0px;
```

```
      display: flex;
```

```
      flex-direction: column;
```

```
    }
```

```
    body{
```

```
      background-image: url("../img/bg6.jpg");
```

```
      background-size: cover;
```

```

    }
    #upload{
        opacity: 0;
    }

</style>
</head>
<body>

<div id="app">
    <div style="position: relative; top:-25px; display: flex; justify-content: flex-end; align-items: center;">
        <!--<h style="margin:auto; font-size: 30px;; font-weight: bold;">Video analysis system</h-->
        <span style=" font-weight: bold; color: rgb(221, 234, 245);">Welcome: {{username}}<el-button size="mini" @click="exit">Logout</el-button></span>
    </div>
    <el-row style="height:800px; flex: 1;" :gutter="20" >
        <el-col :span="12" style="height: 100%;">
            <el-card style="text-align: left; margin-bottom: 20px; background-color: rgba(240, 248, 255, 0.4);">
                <div slot="header" class="clearfix">
                    <i style="color: #6cb5ed; font-size: 25px;" class="el-icon-video-camera-solid"></i>
                    <span style="font-weight: bold; font-size: larger; margin-right: 220px; color:white">Video Play</span>
                    <el-button @click="dialogVisible1 = true" >Upload video</el-button>
                    <el-button type="primary" @click="startDetection">Analyze the video</el-button>
                </div>
                <video id="vid" controls :src="video.path" style="width: 500px;margin-left: 70px;"></video>
                <video id="vidd" controls style="width: 500px; margin-left: 70px;display: none;">
                    <source
src="C:\Users\m's'm\Desktop\video\src\main\resources\static\video\2.mp4" type="video/mp4">
                </video>
            </el-card>

            <el-card style="background-color: rgba(240, 248, 255, 0.4); height: 265px;">
                <div slot="header" class="clearfix">
                    <i style="color: #6cb5ed; font-size: 25px;" class="el-icon-tickets"></i>
                    <span style="font-weight: bold; color:white; font-size: larger;">Video
list</span>

```

```

<!--                                {{video}}-->
                                </div>
                                <el-table max-height="300px" :data="videoList" style=" background:none; width:
100%;height: 160px;" :show-header="false" @row-click="handleRowClick">
                                <el-table-column prop="id" label="日期" width=""></el-table-column>
                                <el-table-column prop="name" label="姓名" width=""></el-table-column>
                                </el-table>
                                </el-card>
</el-col>
<el-col :span="12" style="height: 90%">
    <el-card style="height: 100%;overflow: auto; background-color: rgba(240, 248, 255,
0.4);" >
        <div slot="header" class="clearfix">
            <span style="font-weight: bold;margin-right: 20px;color:white; font-size:
larger;" @click="">Analysis results</span>
            <span id="message"></span>
        </div>
        <div style="padding-bottom: 10px">
            <el-input v-model="searchText" placeholder="Please enter your search
term" style="width: 50%"></el-input>
            <el-button type="primary" @click="search">Search</el-button>
            <el-button id="upload" type="primary" @click="dialogVisible =
true">upload</el-button>
        </div>
        <div>
            <!--Search results show-->
            <el-row>
<!--                                {{pictureList}}-->
                                <el-col :span="8" v-for="pic in pictureList" >
                                    <el-image :src="pic.path" style="height: 100px"></el-image>
                                    <!---->
                                </el-col>
            </el-row>
        </div>
    </el-card>
</el-col>
</el-row>

<!-- Upload -->
    <el-dialog title="upload" :visible.sync="dialogVisible" width="30%">
        <el-upload
            ref="elUpload"
            class="upload-demo"
            action="/picture/uploadImg"

```

```

        :on-success="handleUploadSuccess"
        :before-upload="beforeUpload"
        :on-exceed="handleExceed"
        :limit="3"
        :file-list="fileList"
        :auto-upload="false"
        list-type="picture-card"
    >
        <i class="el-icon-plus"></i>
    </el-upload>
    <div slot="footer" class="dialog-footer">
        <el-button @click="dialogVisible = false">Cancel</el-button>
        <el-button type="primary" @click="submit">Confirm</el-button>
    </div>
</el-dialog>

<el-dialog title="Upload Video" :visible.sync="dialogVisible1" width="30%">
    <el-upload
        ref="elUploadVideo"
        class="upload-demo"
        action="/picture/uploadVideo"
        :on-success="handleUploadSuccess"
        :before-upload="beforeUpload"
        :on-exceed="handleExceed"
        :limit="1"
        :file-list="fileList"
        :auto-upload="false"

    >
        <i class="el-icon-plus"></i>
    </el-upload>
    <div slot="footer" class="dialog-footer">
        <el-button @click="dialogVisible = false">Cancel</el-button>
        <el-button type="primary" @click="submitVideo">Confirm</el-button>
    </div>
</el-dialog>

</div>
<script src="https://cdn.jsdelivr.net/npm/vue/dist/vue.js"></script>
<script src="https://unpkg.com/element-ui/lib/index.js"></script>
<script src="https://ajax.aspnetcdn.com/ajax/jquery/jquery-3.5.1.min.js"></script>
<script src="./js/websocket.js"></script>
<script>
    var app = new Vue({

```

```

el: '#app',
data: {
  videoList: [],
  video: {},
  pictureList: [],
  searchText: "",
  count: 0,
  dialogVisible: false,
  dialogVisible1: false,
  fileList: [],
  username: ""
},
mounted() {
  // load list of video
  this.loadVideoList();
  this.username = localStorage.getItem("username")
},
methods: {
  exit() {
    window.location.href = './login.html';
  },
  startDetection() {

    this.$alert('Being calculated, please wait a moment', 'Tip', {
      confirmButtonText: 'Confirm',
      callback: action => {
        var video = document.getElementById('vid');
var video1 = document.getElementById('vidd');
video1.style.display="block";
video.style.display="none";

      }
    });

    $.ajax({
      type: 'GET',
      url: '/picture/startDetection',
      success: (res) => {
        if (res.code === 0) {
          // return the home page
          this.videoList = []
          this.videoList = res.body
          console.log( this.videoList )
        }
      }
    })
  }
}

```

```

        } else {
            ElementUI.Message.error(res.msg);
        }
    }
    })
},
handleRowClick(row){
    this.video = row;
},
handleUploadSuccess(response, file, fileList) {
},
beforeUpload(file) {
},
handleExceed(files, fileList) {
},
submitVideo(){
    this.$refs.elUploadVideo.submit()
    this.dialogVisible1= false
},
submit() {
    console.log("fileList",this.fileList)
    this.$refs.elUpload.submit()
    this.dialogVisible = false
},
loadVideoList() {
    //Requesting video list data from the backend
    //Here, mock data is used
    $.ajax({
        type: 'GET',
        url: '/picture/getAllVideo',
        success: (res)=>{
            if (res.code === 0) {
                this.videoList = []
                this.videoList = res.body
                console.log( this.videoList )

            } else {
                ElementUI.Message.error(res.msg);
            }
        }
    })
    const mockData = [
    ];
    this.videoList = mockData;

```

```
},  
search() {
```

```
    const postData = {  
        searchText:this.searchText  
        // : this.loginForm.password  
    };  
    $.ajax({  
        type: 'POST',  
        url: '/picture/search',  
        data: postData,  
        success: (res)=>{  
            if (res.code === 0) {  
                this.pictureList = []  
                this.pictureList = res.body  
                console.log( this.pictureList )  
  
            } else {  
                ElementUI.Message.error(res.msg);  
            }  
        }  
    })  
}
```

```
    }  
});
```

```
</script>
```

```
</body>
```

```
</html>
```